

NAMP: A NETWORK-AWARE MODEL PARTITIONING FRAMEWORK FOR CONSTRAINED DEVICES

Fernando Rego @ Fraunhofer Portugal AICOS

João Oliveira @ Fraunhofer Portugal AICOS

Filipe Sousa @ Fraunhofer Portugal AICOS

Luis Almeida @ CISTER / FEUP – University of Porto

Context

- **Internet of Things (IoT)** systems are becoming increasingly common in our daily lives
 - More and more **resource-constrained devices** are being deployed to monitor and control the environment
 - E.g. Smart homes, smart cities, Industry 4.0
- **TinyML** enables Machine Learning (ML) on resource-constrained devices
 - The dependency on cloud services is reduced
 - Enhanced security
 - Cost reduction
 - Low energy consumption

Challenges and Objectives

■ Challenges

- Devices **severely constrained** in computing power, memory and energy consumption
 - ML models usually require a large amount of resources
- Resources available in the **network of devices** are not efficiently utilized
 - Model inference is usually performed in a single device

■ How can all network resources be efficiently utilized to inference ML models?

- Provide mechanisms to **partition ML models** into small deployable services
- Enable **distributed model inference**
- Achieve a **network-aware deployment** of ML models across a network of heterogeneous devices

Related Work

■ Model Partitioning Methods

- Per-layer partitioning
- Graph-based partitioning

■ ML Frameworks

- Popular ML frameworks are unsuitable for IoT systems due to their limited support for constrained devices
 - E.g. TensorFlow [1], PyTorch [2]
- TinyML frameworks consider the limitations of the devices but may require manual partitioning and deployment
 - E.g. TensorFlow Lite for Microcontrollers (TFLM) [3], DIANNE [4]
- Graph-based ML frameworks and tools supports partitioning but usually target large ML models

- None optimizes ML model distribution considering the resources available in a network of constrained devices

High-level Overview of NAMP Framework

■ Model Partitioning Module

- Extract all possible submodels

■ Emulator

- Characterization of each submodel

■ Constraint Generator

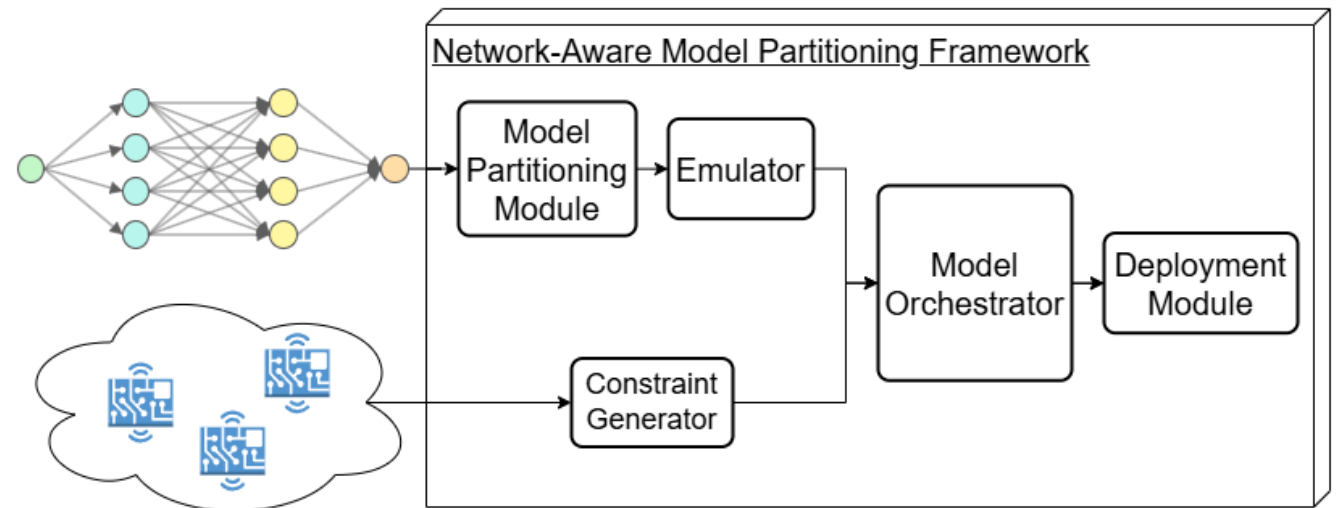
- According to device resource limitations

■ Model Orchestrator

- Pathfinding problem with constraints

■ Deployment Module

- Responsible to deploy the final solution into the network of devices



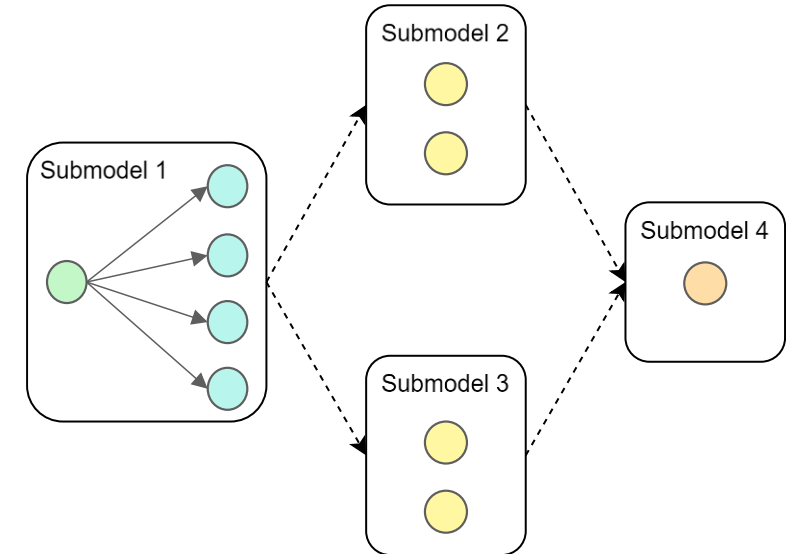
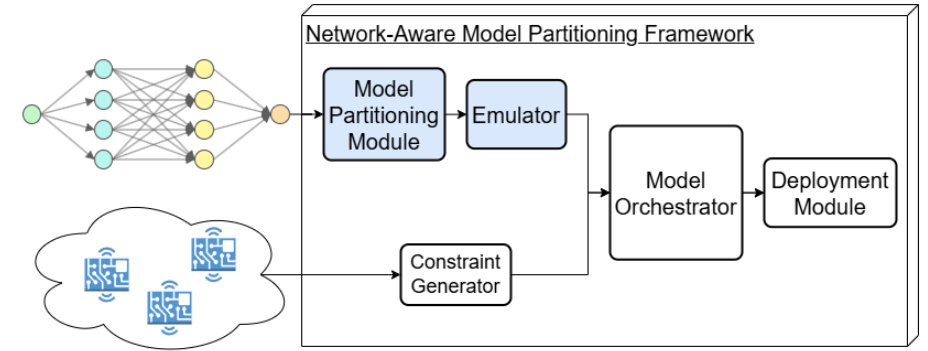
Main Features

■ Model Partitioning Module

- Extract all submodels and convert them to TFLM format
- Model partitioning methods
 - Per-Layer Partitioning
 - Fully Connected Layer Replacement
 - Layer replaced by smaller layers that can be parallelly executed

■ Emulator

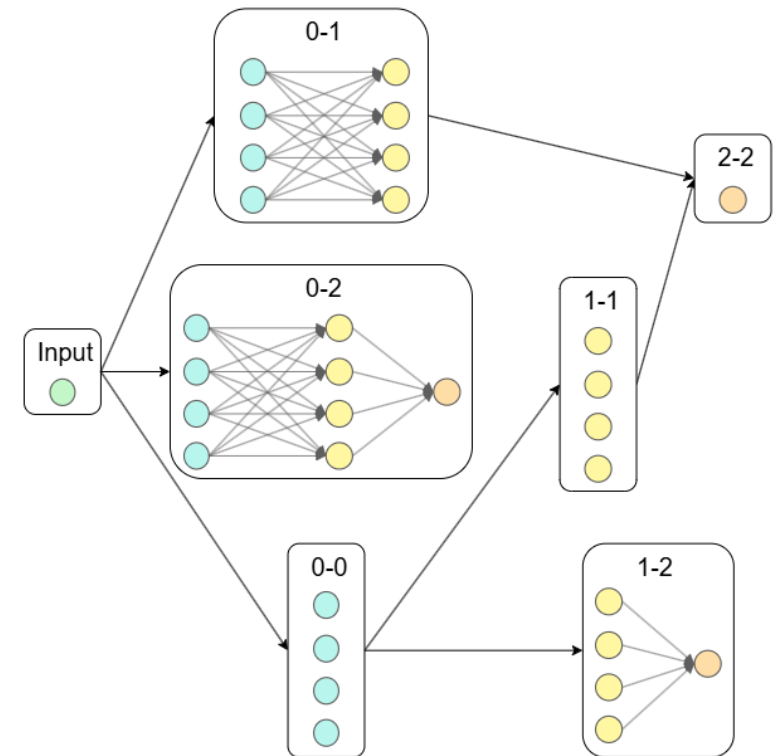
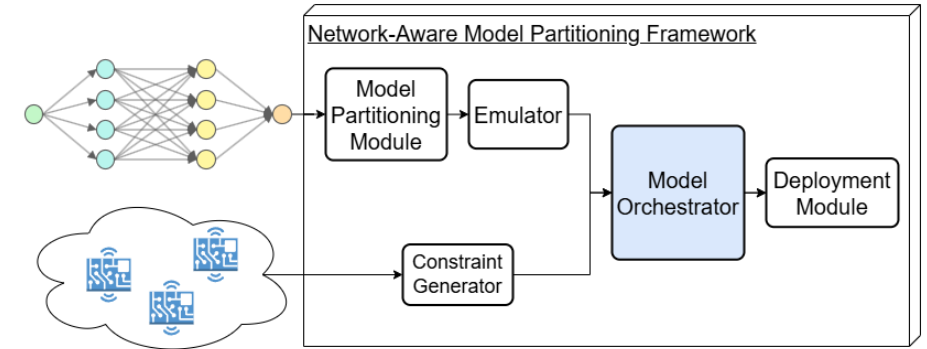
- Emulate a **firmware** application containing the **TFLM interpreter**
- Required to accurately obtain the **tensor arena size** of TFLM
- Retrieve submodel **requirements**
 - E.g. ML model size, number of inputs and outputs



Main Features

■ Model Orchestrator

- Represent the problem with a **graph**
 - Each node represents a submodel
 - Each edge defines the connection between sequential submodels
 - The edge weights follow the implemented Heuristic
- Finding the **final solution**
 - Yen's algorithm [5] to obtain the k shortest path
 - Backtracking algorithm to validate and assign the obtained execution path
 - Final solution must satisfy all constraints



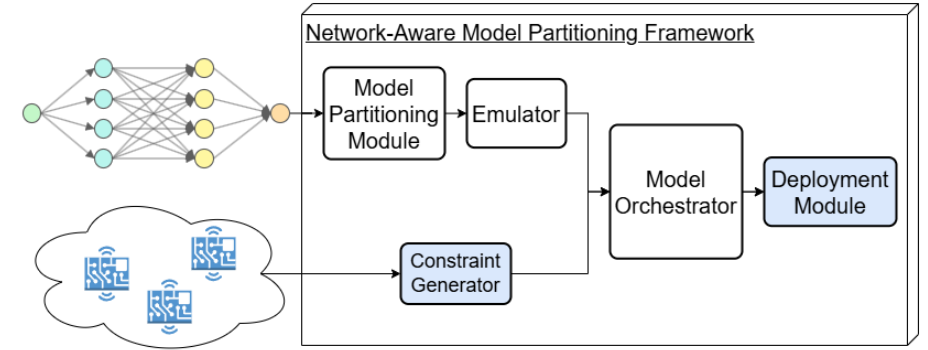
Main Features

■ Constraint Generator

- Retrieve **relevant information** about the **available network resources**
 - E.g. Devices allocatable memory, tensor arena size
- Generate **constraints** that must be **satisfied** by the final solution
 - E.g. To deploy model X , the device must have Y available memory

■ Deployment Module

- Deploys the final NAMP solution in the network of devices
- Current implementation communicates with devices using the CoAP [6] protocol.



Framework Extensibility

■ NAMP Framework extensibility

- Network of heterogenous devices
- Different optimization targets
- NAMP allows the implementation of the following classes:
 - Constraint
 - Device
 - Device Network
 - Heuristic

■ Heuristic current implementation

- Minimize data transfers in the distributed inference

```
1 class Constraint:
2     abstract void init(args)
3     abstract bool validate(model)
4
5 class Device:
6     list[Constraint] constraints
7     abstract void init(args)
8     abstract list[Constraint] collect_constraints()
9     abstract bool validate_constraints(model)
10
11 class DeviceNetwork:
12     list[Device] devices
13     abstract void init(args)
14     abstract list[Device] collect_devices()
15     abstract bool deploy(model_assignments)
16
17 class Heuristic:
18     abstract void set_weights(graph, devices)
```

NAMP Framework API

Experimental Setup and ML Model Specification

- A network of 15 devices was emulated using **IoTNetEmu** [7] and **QEMU** [8] capabilities
 - Cortex-M3 nodes
 - Nodes are connected via Linux Bridges
- A **Firmware Application** was developed on top of **Zephyr** RTOS [9] for the devices
 - Communicates via CoAP protocol
 - Receive input(s) and ML model(s)
 - Transmit inference output(s)
 - Enables model inference
 - Employs the TFLM interpreter

LeNet-based ML Model

Submodel	Layer Type	Model Size (B)	Tensor Arena (B)	Number of Inputs	Number of Outputs
0-0	Convolutional	1776	17536	784	3456
1-1	Pooling	912	17760	3456	864
2-2	Convolutional	10808	8192	864	1024
3-3	Pooling	912	5600	1024	256
4-4	Flatten	968	1496	256	256
5-5	Fully Connected	124436	1984	256	120
6-6	Flatten	952	1408	120	120
7-7	Fully Connected	41732	1296	120	84
8-8	Flatten	952	1120	84	84
9-9	Fully Connected	4672	976	84	10
0-9	Entire Model	181904	19200	784	10

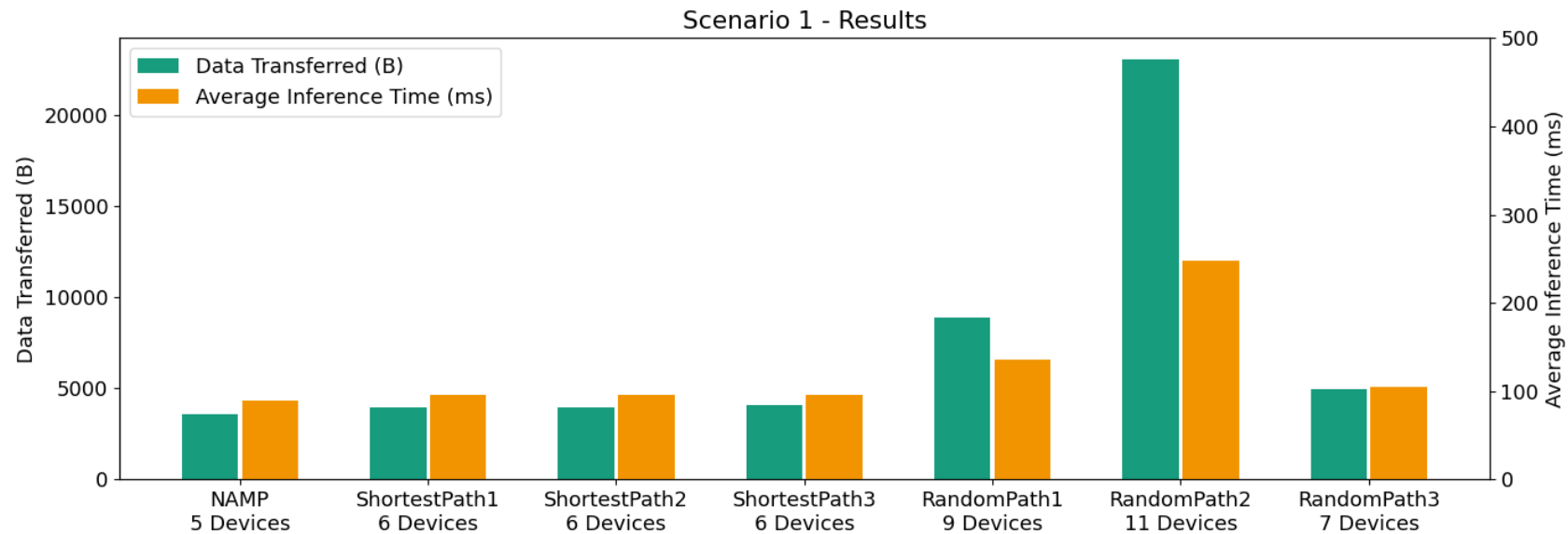
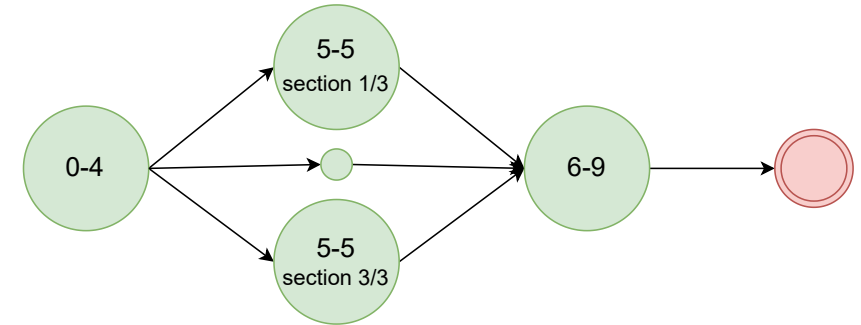
<https://www.kaggle.com/models/fernandorego0/lenet>

Results – Scenario 1

■ Devices with **84 KiB** of available memory

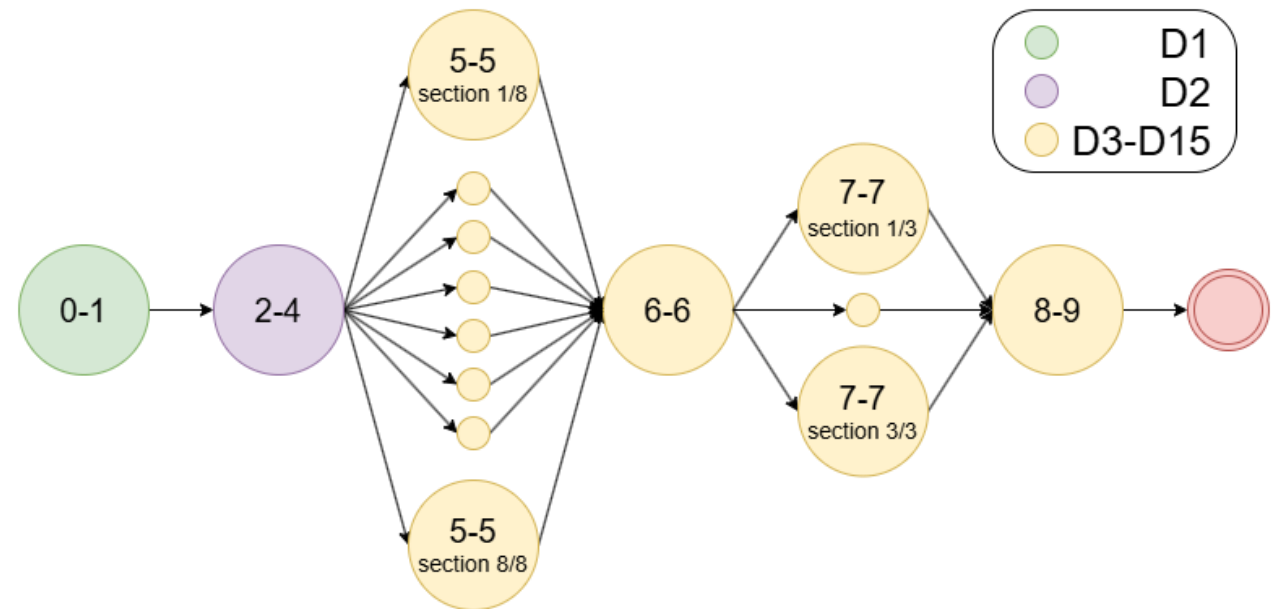
■ Heap Memory – 64 KiB

■ Tensor Arena – 20 KiB



Results – Scenario 2

- Device **D1** with **32 KiB** of available memory
 - Heap Memory – 12 KiB
 - Tensor Arena – 20 KiB
- Device **D2** with **32 KiB** of available memory
 - Heap Memory – 20 KiB
 - Tensor Arena – 12 KiB
- Devices **D3-D15** with **24 KiB** of available memory
 - Heap Memory – 20 KiB
 - Tensor Arena – 4 KiB



Conclusion and Future Work

■ NAMP Framework for Constrained Devices

- Provide mechanisms to partition ML models into **smaller models** that can be deployed as **services**
- Achieved a **network-aware model orchestration** efficiently utilizing the available network resources

■ Future Work

- Integration of NAMP with an IoT orchestration system enabling **dynamic model redeployment** in response to network resource changes
- Expand available **orchestration algorithms** and optimization targets
- Explore partitioning mechanisms to different types of ML Layers

Table of Contents

- Context
- Challenges and Objectives
- Related Work
- Network-Aware Model Partitioning Framework
 - High-level Overview
 - Main features
 - Framework Extensibility
- Evaluation
 - Experimental Setup
 - Results
- Conclusion and Future Work

References

- [1] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, M. Kudlur, J. Levenberg, R. Monga, S. Moore, Derek G. Murray, B. Steiner, P. Tucker, V. Vasudevan, P. Warden, M. Wicke, Y. Yu, and X. Zheng. 2016. TensorFlow: a system for large-scale machine learning. In Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation (Savannah, GA, USA) (OSDI'16). USA, 265–283. <https://doi.org/10.48550/arXiv.1605.08695>
- [2] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala. 2019. PyTorch: an imperative style, high-performance deep learning library. Red Hook, NY, USA. <https://doi.org/10.48550/arXiv.1912.01703>
- [3] Robert David et al. 2021. TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems. In Proceedings of Machine Learning and Systems, 800–811. <https://doi.org/10.48550/arXiv.2010.08678>
- [4] Elias D. Coninck, T. Verbelen, B. Vankeirsbilck, S. Bohez, S. Leroux, and P. Simoens. 2015. DIANNE: Distributed Artificial Neural Networks for the Internet of Things. In Proceedings of the 2nd Workshop on Middleware for Context-Aware Applications in the IoT. Vancouver, BC, Canada. <https://doi.org/10.1145/2836127.2836130>

References

- [5] Jin Y. Yen. 1970. An algorithm for finding shortest routes from all source nodes to a given destination in general networks. In Quarterly of Applied Mathematics 27.4, pp. 526–530. <https://doi.org/10.1090/qam/253822>
- [6] Zach Shelby, Klaus Hartke, and Carsten Bormann. 2014. The Con-strained Application Protocol (CoAP). RFC 7252. <https://doi.org/10.17487/RFC7252>
- [7] João Oliveira, Filipe Sousa, and Luis Almeida. 2023. IoTNetEMU - A Framework to Emulate and Test IoT Applications. In Proceedings of the 19th ACM International Symposium on QoS and Security for Wireless and Mobile Networks (Q2SWinet '23). Association for Computing Machinery, New York, NY, USA, 33–37. <https://doi.org/10.1145/3616391.3622774>
- [8] Software Freedom Conservancy. QEMU. <https://www.qemu.org>
- [9] Linux Foundation. Zephyr Project. <https://zephyrproject.org>